

Lesson 7 — Strengthening Logic & Thinking Through Practice

Introduction

At this stage, you are no longer just learning how syntax works — you are training your brain to **think logically, step by step**, and to stay calm when a problem does not immediately make sense. This lesson focuses on strengthening core programming thinking skills such as iteration, conditions, state tracking, and debugging.

These exercises are intentionally simple in appearance but powerful in effect. They force you to reason carefully, follow the flow of execution, and understand *why* a solution works instead of memorizing patterns blindly. This is where real progress happens.

While working on these problems, try to solve them **on your own first**, even if it feels slow or frustrating. That struggle is not a weakness — it is the process that builds strong problem-solving skills.

Exercise 1 — Alone (Logic + Loop)

Count Even and Odd Numbers

Problem:

Write a function that takes a list of integers and returns how many numbers are even and how many are odd.

Example:

Input: [1, 2, 3, 4, 6]

Output: Even: 3, Odd: 2

What this trains:

- Modulo reasoning
 - Using multiple counters
 - Careful condition checking
-

Exercise 2 — With Hint (Edge Cases)

Second Largest Number

Problem:

Write a function that returns the second largest number in a list of integers.

Rules:

- Assume the list has at least two elements
- Do **not** use sorting

Example:

Input: `[5, 1, 9, 7]`

Output: `7`

Hint:

- Track two values: `largest` and `second_largest`
- Update carefully when a new maximum appears

What this trains:

- Tracking multiple states
 - Logical ordering
 - Avoiding shortcuts
-

Exercise 3 — Alone (Pattern Recognition)

Detect Duplicates

Problem:

Write a function that returns `True` if a list contains any duplicate value, otherwise returns `False`.

Examples:

`[1, 2, 3, 4] → False`

`[1, 2, 3, 2] → True`

What this trains:

- Comparison logic
 - Early stopping
 - Real-world data validation thinking
-

Exercise 4 — With Hint (Step-by-Step Reasoning)

Running Total

Problem:

Given a list of numbers, return a new list where each element is the sum of all previous elements (including itself).

Example:

Input: [1, 2, 3, 4]

Output: [1, 3, 6, 10]

Hint:

- Keep a variable called `current_sum`
- Add each number to it and append the result

What this trains:

- Accumulation logic
 - Understanding how state evolves inside loops
-

Exercise 5 — Debugging (Logic)

Find First Negative Number

Below is a buggy function intended to return the first negative number in a list, or `None` if there is none.

```
def first_negative(lst):
```

```
for i in range(len(lst)):
    if lst[i] < 0:
        return lst[i]
    else:
        return None
```

What to do:

- Explain why this code fails for many inputs
- Fix the logic
- Show two test cases where the corrected version works

Hints:

- Think about when `return` should happen
- Should `None` be returned inside the loop or after it?

Conclusion

Logic is not built by memorizing code, but by **repeatedly facing problems, making mistakes, and correcting them**. Each exercise you solve strengthens your ability to reason, focus, and break complex problems into manageable steps.

Activities that feel like games — such as logic puzzles, IQ challenges, strategy games, or problem-solving competitions — are excellent complements to coding practice because they train the same mental muscles: patience, analysis, and structured thinking.

Treat each exercise as a puzzle, not a test. Struggling is part of the learning process, and every honest attempt brings you closer to thinking clearly, confidently, and independently as a programmer.